Module 5

Ensemble learning

Dr. Pratik Oak
Department of CSE AIML, GIT

Different Classifiers

- Performance
  - None of the classifiers is perfect
  - Complementary
    - Examples which are not correctly classified by one classifier may be correctly classified by the other classifiers
- Potential Improvements?
  - Utilize the complementary property

Real life example

- **Let's take a real example to build the intuition.**

- Suppose, you want to invest in a company XYZ. You are not sure about its performance though.
- So, you look for advice on whether the stock price will increase by more than 6% per annum or not?
- You decide to approach various experts having diverse domain experience

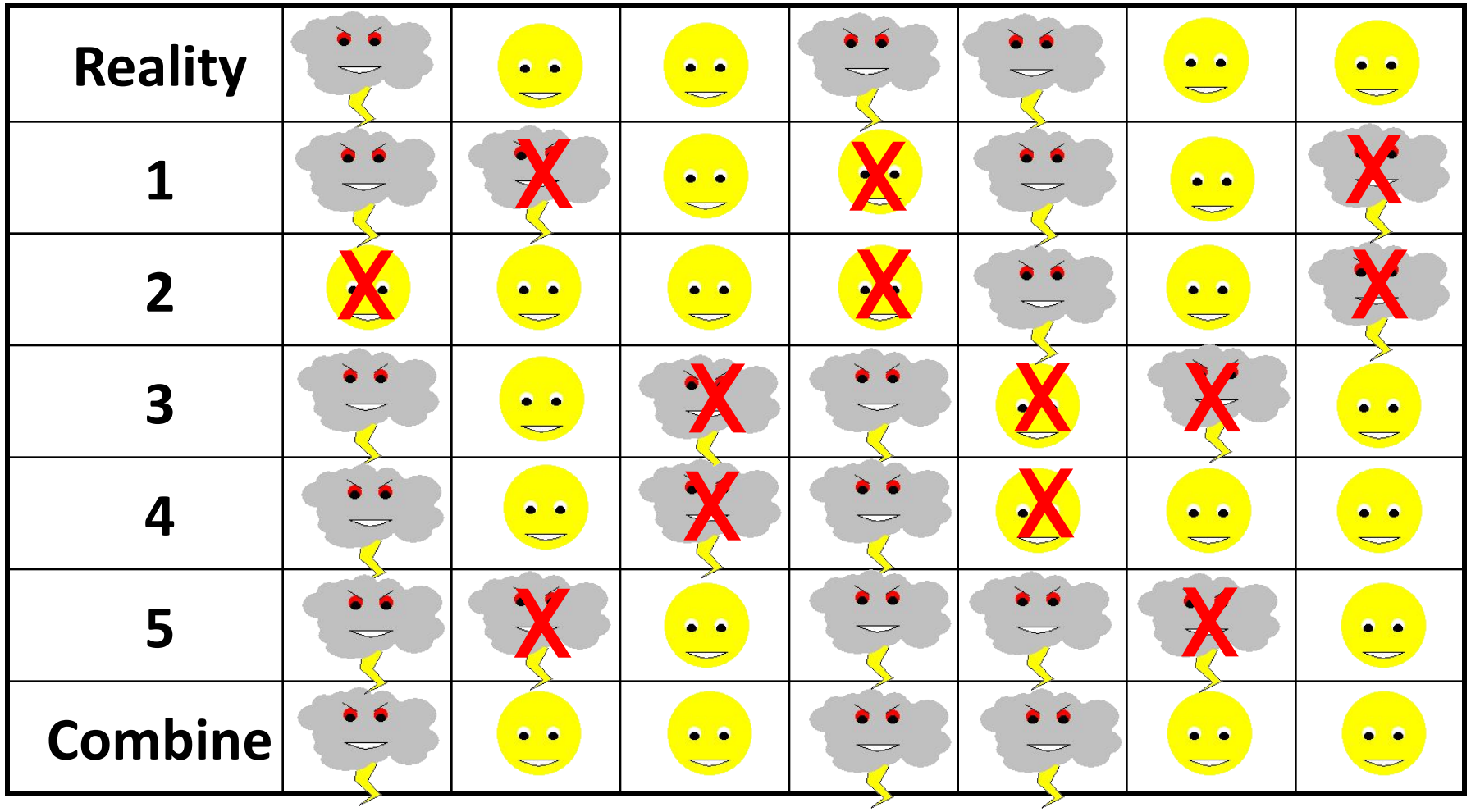
The survey prediction

- Employee of Company XYZ: – In the past, he has been right 70% times.
- Financial Advisor of Company XYZ: – In the past, he has been right 75% times.
- Stock Market Trader: – In the past, he has been right 70% times.
- Employee of a competitor: – In the past, he has been right 60% times.
- Market Research team in the same segment: – In the past, he has been right 75% times.
- Social Media Expert: – In the past, he has been right 65% times
- Given the broad spectrum of access you have, you can probably combine all the information and make an informed decision.

Conclusion

- In a scenario when all the 6 experts/teams verify that it's a good decision(assuming all the predictions are independent of each other), you will get a combined accuracy rate of 1 - (30% . 25% . 30% . 40% . 25% . 35%) = 1 - 0.07875 = 99.92125%
- The assumption used here that all the predictions are completely independent is slightly extreme as they are expected to be correlated. However, you can see how we can be so sure by combining various forecasts together.
- Well, Ensemble learning is no different. (But the accuracy of the ensemble model cannot be directly calculated by multiplying the individual accuracies of the models together. The accuracy of the ensemble model depends on the specific predictions made by each model and how they align with the true labels in the dataset.)

Example: Weather Forecast



Introduction to Ensemble Learning

- Ensemble learning is a powerful technique in machine learning where multiple (low performing) models are combined to produce better predictive performance than any individual model alone.
- The motivation behind ensemble learning is to harness the collective wisdom (by combining the predictions) of multiple models to improve prediction accuracy, reduce overfitting, and increase robustness.
- Certain models do well in modeling one aspect of the data, while others do well in modeling another
- One of the key principles of ensemble learning is to ensure diversity among the base models.
- Diversity can be achieved by using different algorithms, different subsets of the training data, or different hyperparameters for each model to capture different aspects of the data and reduce the risk of making errors due to biases in individual models.

- Ensemble methods can parallelize by allocating each base learner to different-different machines.
- Finally, Ensemble learning methods can be thought as meta-algorithms that combine several machine learning methods into a single predictive model to increase performance.
- Ensemble-based models can be incorporated in both of the two scenarios, i.e., when data is of large volume and when data is too little.
- Ensemble methods can decrease variance using bagging approach, decrease bias using a boosting approach, or improve predictions using stacking approach.

- Ensemble learning: instead of creating one strong classifier, we create a huge set of weak classifiers, then we combine their outputs into one final decision
  - According to Concordet's theorem, under proper conditions we can expect that the ensemble model can attain an error rate that is arbitrarily close to zero
  - While creating a lot of weak classifiers is hopefully a much easier task than to create one strong classifier

Averaging reduces the model variance

$$Var(\overline{X}) = \frac{Var(X)}{N}$$

(when predictions are **independent**)

Proof: Averaging reduces the model variance

Consider M weak learners, each making independent predictions for an input x. The prediction of the i-th weak learner can be expressed as:

$$\mathbf{f_i(x) = Y - B(x) + \varepsilon_i}$$

where:

Y is the true value (ground truth)
B(x) is the bias of the weak learner.
ε_i is the error (random noise) in the prediction.

Assuming that:

Each weak learner has the same bias B(x).
The errors ε_i are independent and identically distributed with variance σ^2: $Var(\varepsilon_i) = \sigma^2$

The ensemble model averages the predictions of M weak learners:

$$\bar{f}(x) = (1/M) \sum_{1}^{M} f_i(x)$$

Substituting $f_i(x) = Y - B(x) + \varepsilon_i$, we get:

$$\bar{f}(x) = (1/M) \sum_{1}^{M} (Y - B(x) + \varepsilon_i)$$

Since Y and B(x) are constants across models, they remain unchanged:

$$\bar{f}(x) = Y - B(x) + (1/M) \sum_{1}^{M} \varepsilon_i$$

Since bias remains unaffected, we focus on the variance term:

$$\mathrm{Var}(\bar{f}(x)) = \mathrm{Var}\left((1/M) \sum_{1}^{M} \varepsilon_i \right)$$

Using the variance property:

$$\mathrm{Var}(aX) = a^2\, \mathrm{Var}(X)$$

we get:

$$\mathrm{Var}(\bar{f}(x)) = (1/M^2) \sum_{1}^{M} \mathrm{Var}(\varepsilon_i)$$

Since each weak learner has variance σ^2, we get:

$$\mathrm{Var}(\bar{f}(x)) = (1/M^2) \times M\, \sigma^2 = \sigma^2/M$$

Thus, the variance of the ensemble model is σ^2/M, which is M times lower than the variance of an individual weak learner (σ^2).

Benefits:

1. **Improved Accuracy:** Ensemble methods often outperform individual models by reducing variance and capturing complementary patterns in the data.
2. **Robustness:** Ensemble methods are less sensitive to noise and outliers in the data (reduces the spread or dispersion of the predictions), making them more robust in real-world scenarios.
3. **Generalization:** By combining diverse models, ensemble methods can generalize better to unseen data and perform well across different datasets and domains.

Challenges:

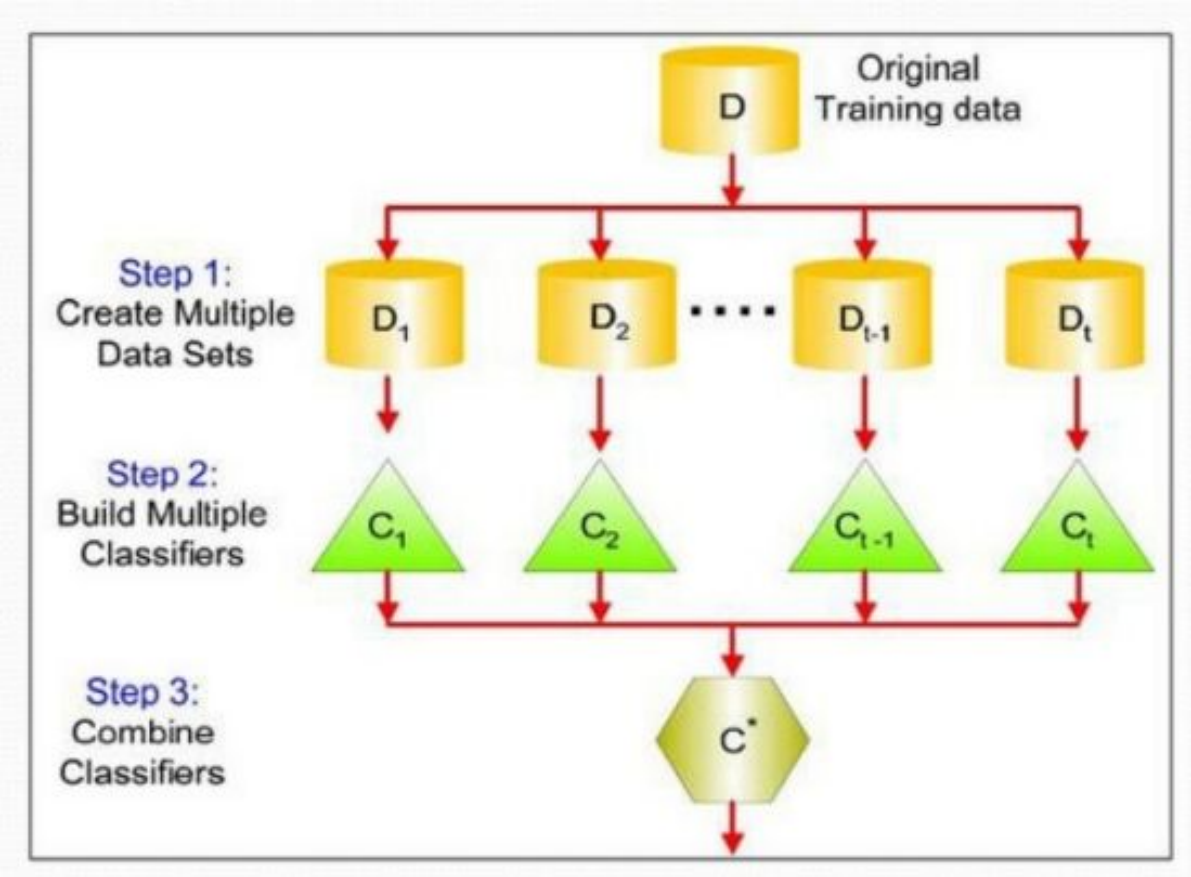
1. **Computational Complexity:** Ensemble methods can be computationally expensive, especially when training large ensembles or using complex base models.
2. **Interpretability:** The predictions of ensemble methods can be more challenging to interpret compared to individual models, especially in stacking where multiple models are combined hierarchically.

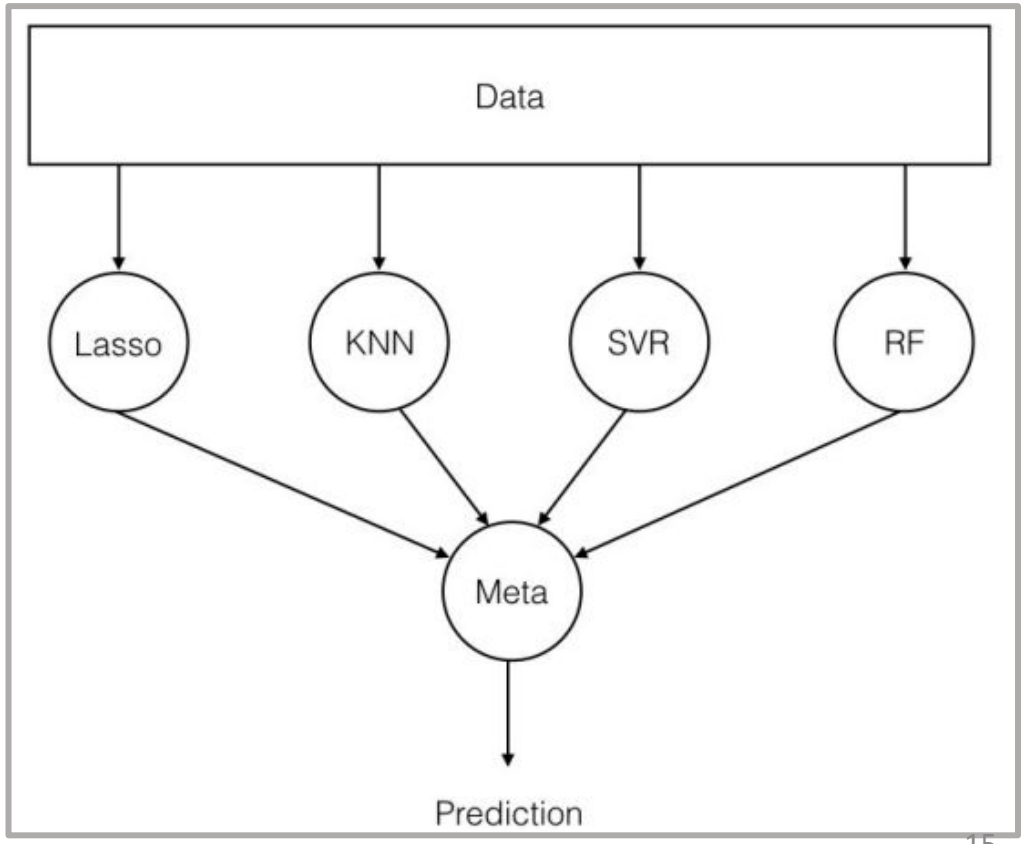
Basic Ensemble Structure

Combine all “weak” learners to form an ensemble

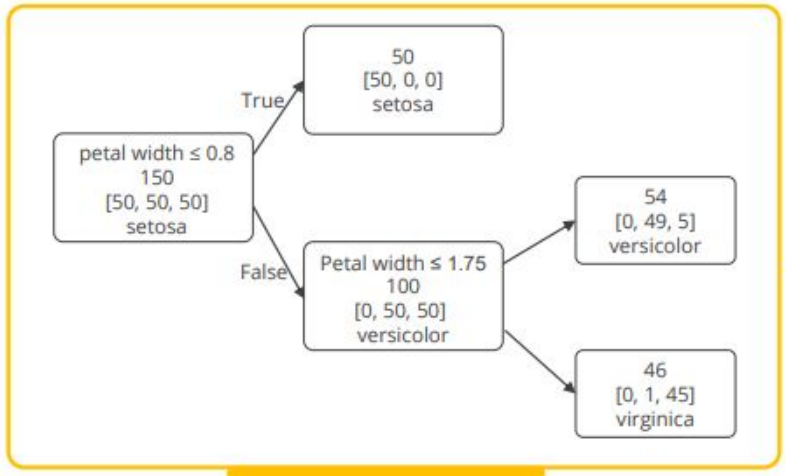
OR

Create an ensemble of well-chosen strong and diverse models

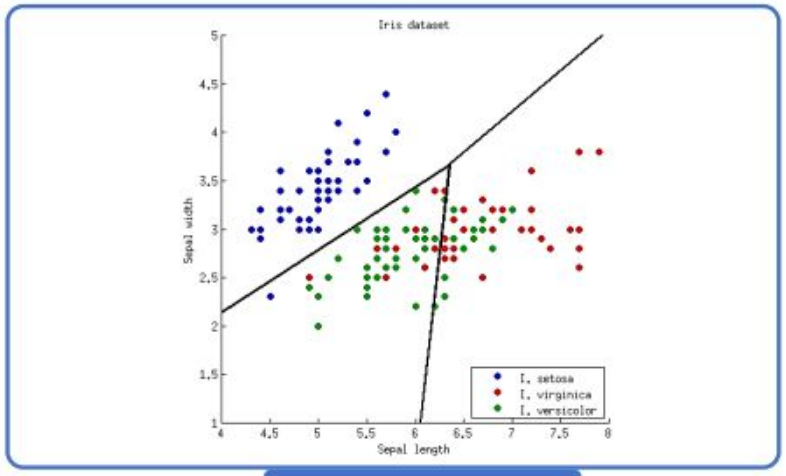




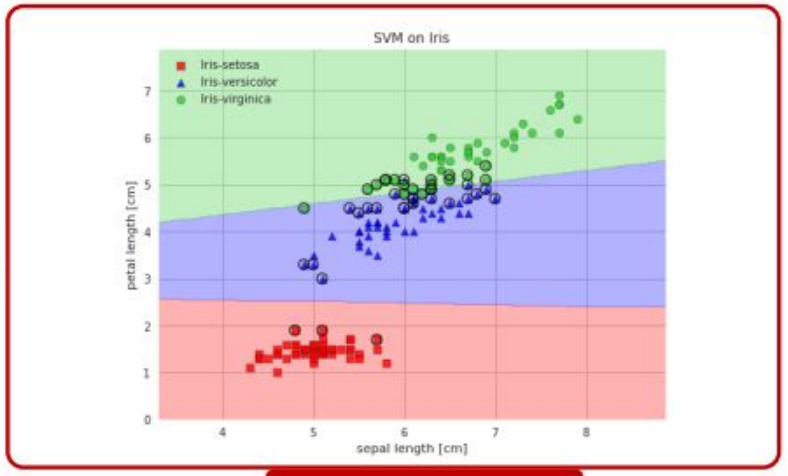
Averaging



Decision Tree



Logistic Regression

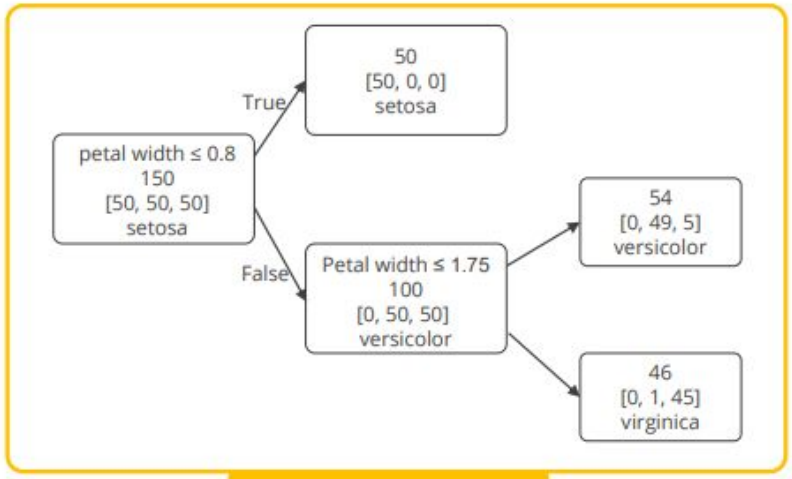


SVM

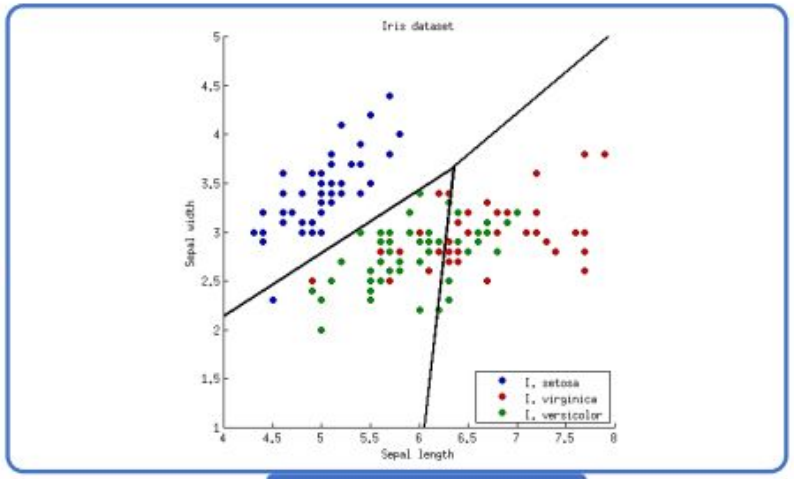
Equal weights are assigned to different models

$$P = \frac{p1 + p2 + p3}{3}$$

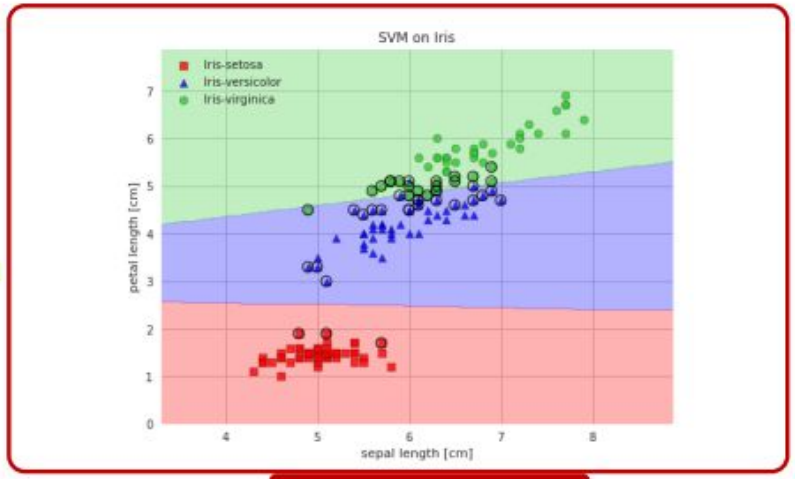
Weighted Averaging



Decision Tree



Logistic Regression



SVM

50

35

15

$w1 + w2 + w3 = 1$

Each model is assigned a different weight

$P = w1 * p1 + w2 * p2 + w3 * p3$

Common Ensemble Methods

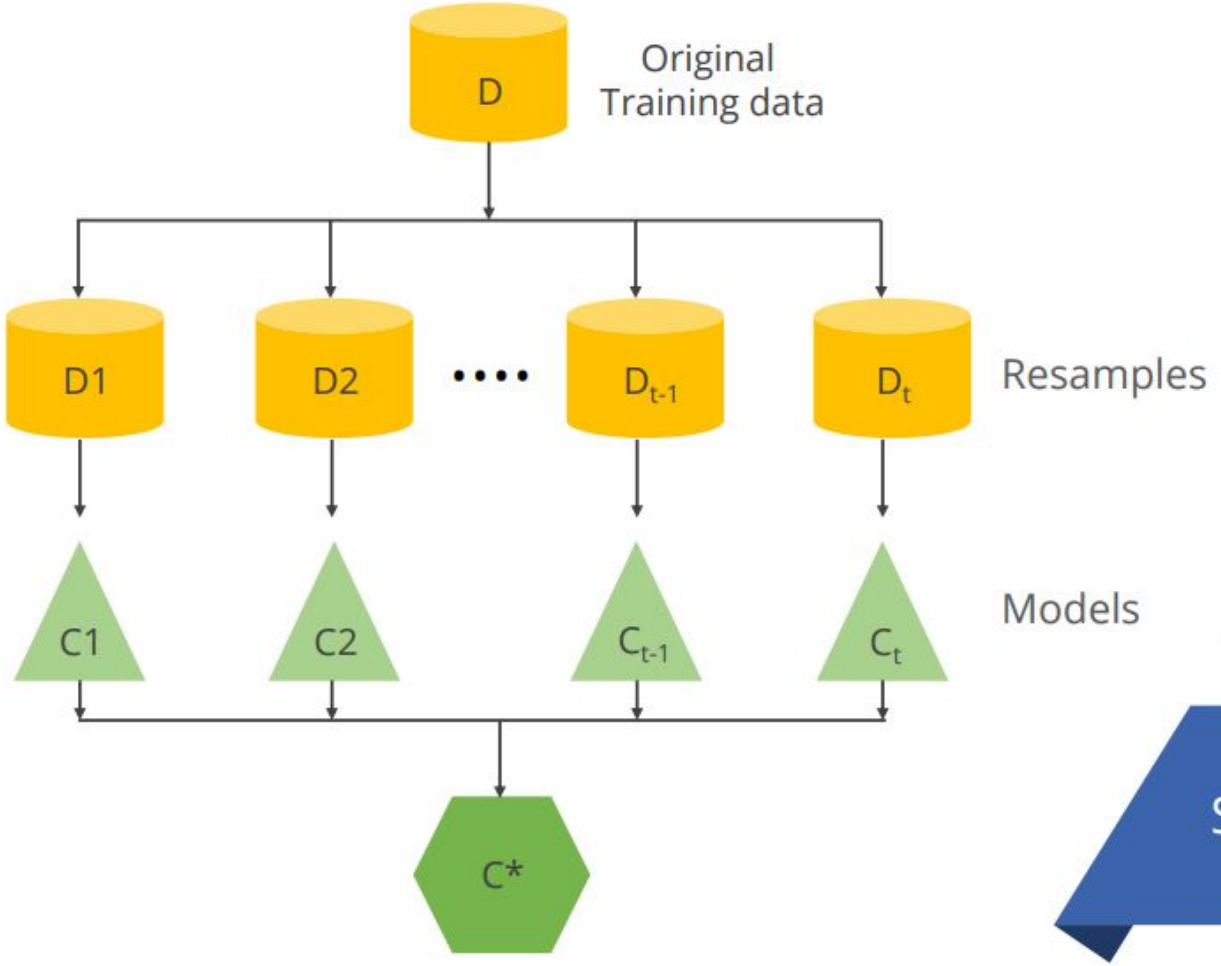
1. **Voting (Majority voting)**: In a voting ensemble, each base model makes a prediction, and the final prediction is determined by a majority vote (for classification) or averaging (for regression).
2. **Bagging (Bootstrap Aggregating)**: Bagging involves training multiple base models on different bootstrap samples (random subsets with replacement) of the training data and averaging their predictions.
3. **Random Forest:** Random Forest is an ensemble method based on bagging that specifically uses decision trees as the base models. In addition to sampling the training data, random forest also randomly selects a subset of features at each split, which helps decorrelate the trees and reduce overfitting.
4. **Boosting**: Boosting algorithms train a sequence of weak learners (models that perform slightly better than random chance) where each subsequent model focuses on correcting the errors of its predecessors.
5. **Stacking**: Stacking combines the predictions of multiple base models by training a meta-model (often a simple linear model) on the outputs of the base models.

What is Bootstrapping?

- It is a **sampling technique** used to generate multiple datasets from the original dataset.
- Given a dataset of size **N**, randomly select **N** samples **with replacement**.
- Some samples may appear **multiple times**, while others may be excluded.
- This process is repeated to create **multiple bootstrapped datasets**.
- It helps estimate model uncertainty, works well for small datasets by generating variations and forms the foundation for bagging.

1. Bagging (Bootstrap Aggregating)

Bagging or bootstrap aggregation **reduces variance** of an estimate by taking mean of multiple estimates



STEP 1 Create randomly sampled datasets of the original training data (bootstrapping)

STEP 2 Build and fit several classifiers to each of these diverse copies

STEP 3 Take the average of all the predictions to make final overall prediction

$$f(x) = 1/M \sum_{m=1}^{M} f_m(x)$$

Bagging (Bootstrap Aggregating) continued…

- Bagging involves training multiple models independently on different subsets of the training data and then combining their predictions to obtain the final output.
- Bagging is particularly effective when used with high-variance models that are prone to overfitting, such as decision trees.

•Working:

1. **Bootstrap Sampling**:
   - ✔ Bagging begins by creating multiple subsets of the training data through bootstrap sampling.
   - ✔ Bootstrap sampling involves randomly selecting samples from the original training dataset with replacement.
   - ✔ This means that some instances may be sampled multiple times, while others may not be sampled at all.
   - ✔ Each subset is typically of the same size as the original training dataset.

Bagging (Bootstrap Aggregating) continued…

2.Base Model Training:

- ✔ Once the bootstrap samples are generated, a base model (e.g., decision tree) is trained independently on each subset of the training data.
- ✔ Since each subset contains different instances and may have duplicates, each base model learns slightly different patterns from the data.

3.Prediction Aggregation:

- ✔ After training the base models, bagging combines their predictions to make the final prediction.
- ✔ For regression tasks, the predictions of the base models are typically averaged, while for classification tasks, the final prediction is determined by majority voting.

Bagging and Bootstrapping

Bagging

A technique for reducing the variance of an estimated prediction function

For classification, a committee of trees each cast a vote for the predicted class

Bootstrapping

Randomly draws datasets with replacement from the training data

Each sample is of the same size as the original training set

Advantages of Bagging:

- **Reduced Variance**: Bagging reduces the variance of the final model. This helps to mitigate overfitting and improve generalization performance.
- **Increased Robustness**: Bagging makes the model more robust to noise and outliers in the training data. Since each base model is trained on a subset of the data, they may not be influenced by individual noisy instances as much.
- **Parallelizable**: Since each base model is trained independently, bagging is highly parallelizable, making it suitable for distributed computing and speeding up training on multicore processors or distributed computing frameworks.

- Key elements of bagging:
  - ✔ Bootstrap samples
  - ✔ Unpruned decision trees fit on each sample
  - ✔ Simple voting or averaging of predictions

- Many popular ensemble algorithms are based on this approach:
  - ✔ Bagged decision trees (canonical bagging)
  - ✔ Random Forest
  - ✔ Extra Trees

Bagging vs Bootstrapping

Feature	Bootstrapping	Bagging
Purpose	Sampling technique	Ensemble learning method
Key Idea	Create new datasets by resampling	Train multiple models on bootstrapped datasets
Model Training?	No (used for data generation)	Yes (aggregates multiple models)
Reduces Variance?	No (only modifies data)	Yes (by combining models)

2. Random Forest

- One of the most popular implementations of bagging is the Random Forest algorithm.
- In Random Forest, the base models are **decision trees**.
- Instead of training multiple instances of the same decision tree algorithm, multiple decision trees are trained.
- An additional randomness and diversity is introduced into the ensemble by randomly selecting a subset of features (columns) of the training data at each split of the tree.
- During prediction, the output of all individual trees is combined (e.g., by averaging or voting) to produce the final prediction.

Random Forest continued…

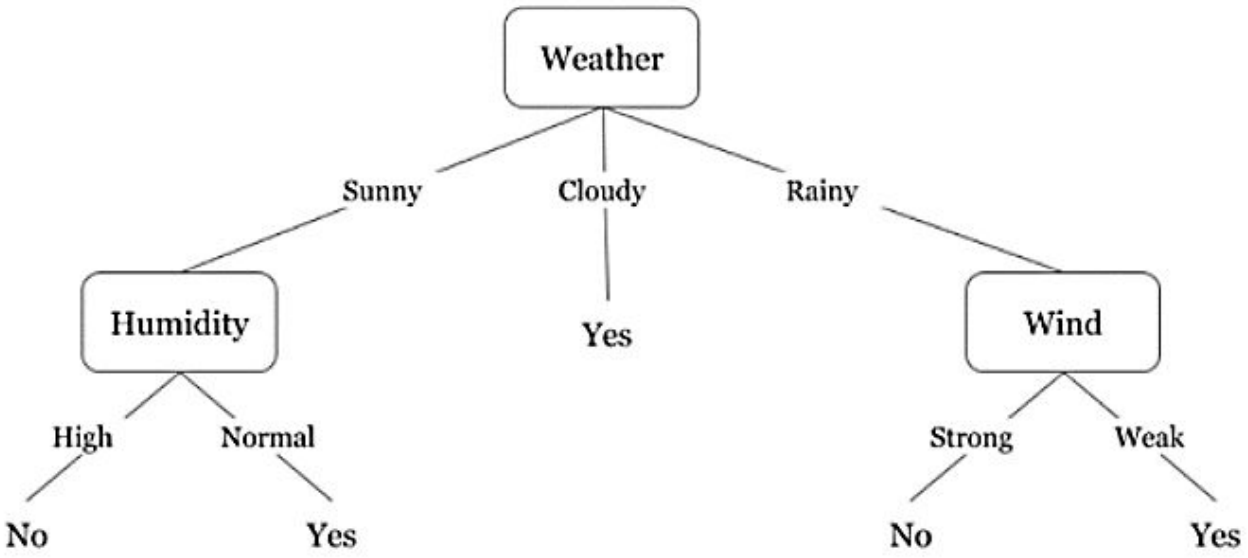
- Random Forests are effective at reducing overfitting, handling high-dimensional data, and providing robust predictions.
- Bagging typically has few hyperparameters to tune, such as the number of base models (often referred to as "n_estimators" in sklearn's implementation) and the size of the bootstrap samples.
- The number of base models should be large enough to capture diverse patterns in the data without overfitting.
- They are widely used in classification and regression tasks and are known for their simplicity and effectiveness.
- Bagging is a general technique that can be applied to various learning algorithms, while Random Forests specifically apply bagging to decision trees.

Decision Tree

….. a quick revision

Decision trees are upside down which means the root is at the top and then this root is split into various several nodes. Decision trees are nothing but a bunch of if-else statements in layman terms. It checks if the condition is true and if it is then it goes to the next node attached to that decision.

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No



We see that if the *weather is cloudy* then we must go to play. Why didn't it split more? Why did it stop there?

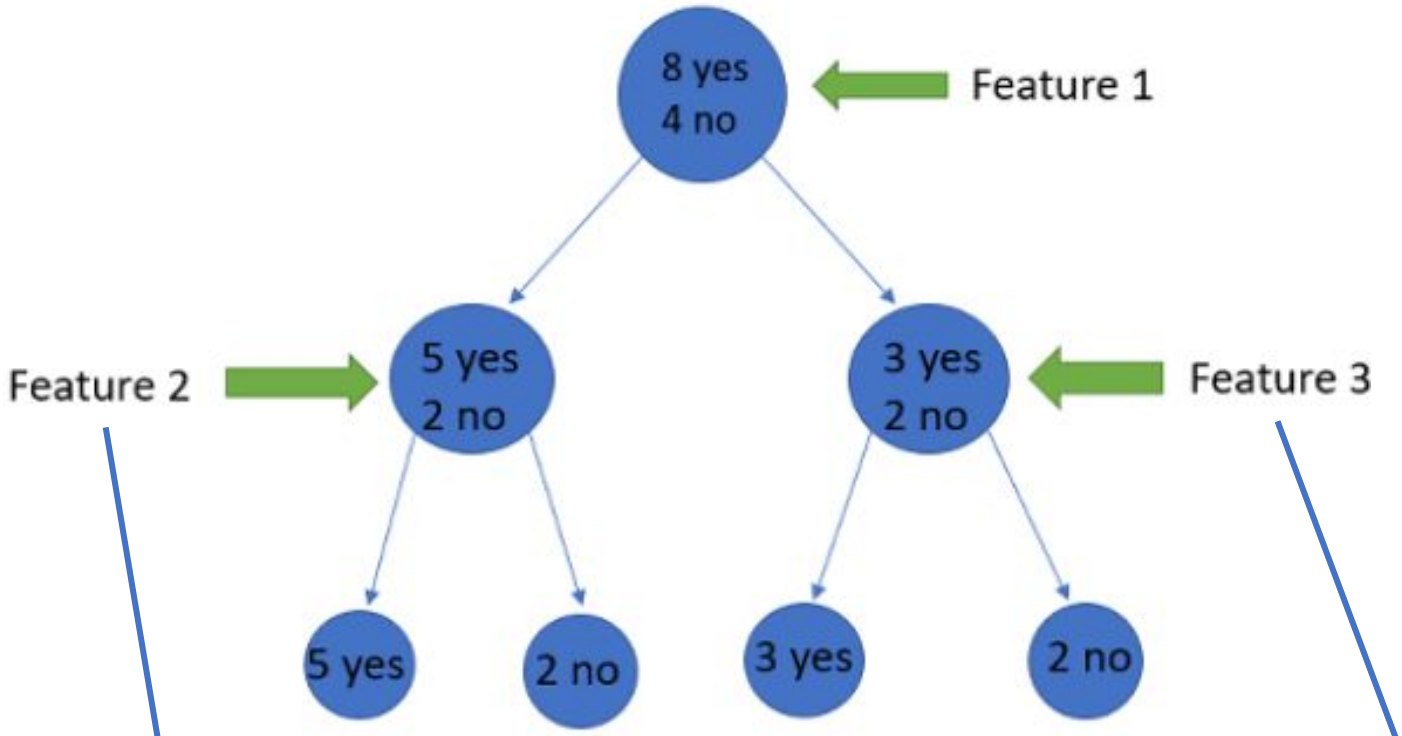
- In simple terms, the output for the training dataset is always yes for cloudy weather, since there is no disorderliness here, we don’t need to split the node further.
- The goal of machine learning is to decrease uncertainty or disorders from the dataset and for this, we use decision trees.
- How do I know what should be the root node?
- What should be the decision node?
- When should I stop splitting?
- To decide this, there is a metric called “Entropy” which is the amount of uncertainty in the dataset.

Entropy

- Entropy is nothing but the uncertainty in our dataset or measure of disorder.
- The formula for Entropy is: $E(S) = -p_{(+)}\log p_{(+)} - p_{(-)}\log p_{(-)}$
  - p_+ is the probability of positive class
  - p_- is the probability of negative class
  - S is the subset of the training example
- Entropy basically measures the impurity of a node. Impurity is the degree of randomness; it tells how random our data is.
- A **pure sub-split** means that either you should be getting “yes”, or you should be getting “no”.
- In order to make a decision tree, we need to calculate the impurity of each split, and when the purity is 100%, we make it as a leaf node.

Jensen's inequality states that for a **concave function** $f(x)$ and probabilities p_i:

$$\sum_{i=1}^{n} p_i f(x_i) \leq f\left(\sum_{i=1}^{n} p_i x_i\right)$$



$$\Rightarrow \; -\left(\frac{5}{7}\right)log_2\left(\frac{5}{7}\right) - \left(\frac{2}{7}\right)log_2\left(\frac{2}{7}\right)$$

$$\Rightarrow \; -(0.71 * -0.49) - (0.28 * -1.83)$$

$$\Rightarrow \; -(-0.34) - (-0.51)$$

$$\Rightarrow \; 0.34 + 0.51$$

$$\Rightarrow \; 0.85$$

$$\Rightarrow \; -\left(\frac{3}{5}\right)log_2\left(\frac{3}{5}\right) - \left(\frac{2}{5}\right)log_2\left(\frac{2}{5}\right)$$

$$\Rightarrow \; -(0.6 * -0.73) - (0.4 * -1.32)$$

$$\Rightarrow \; -(-0.438) - (-0.528)$$

$$\Rightarrow \; 0.438 + 0.528$$

$$\Rightarrow \; 0.966$$

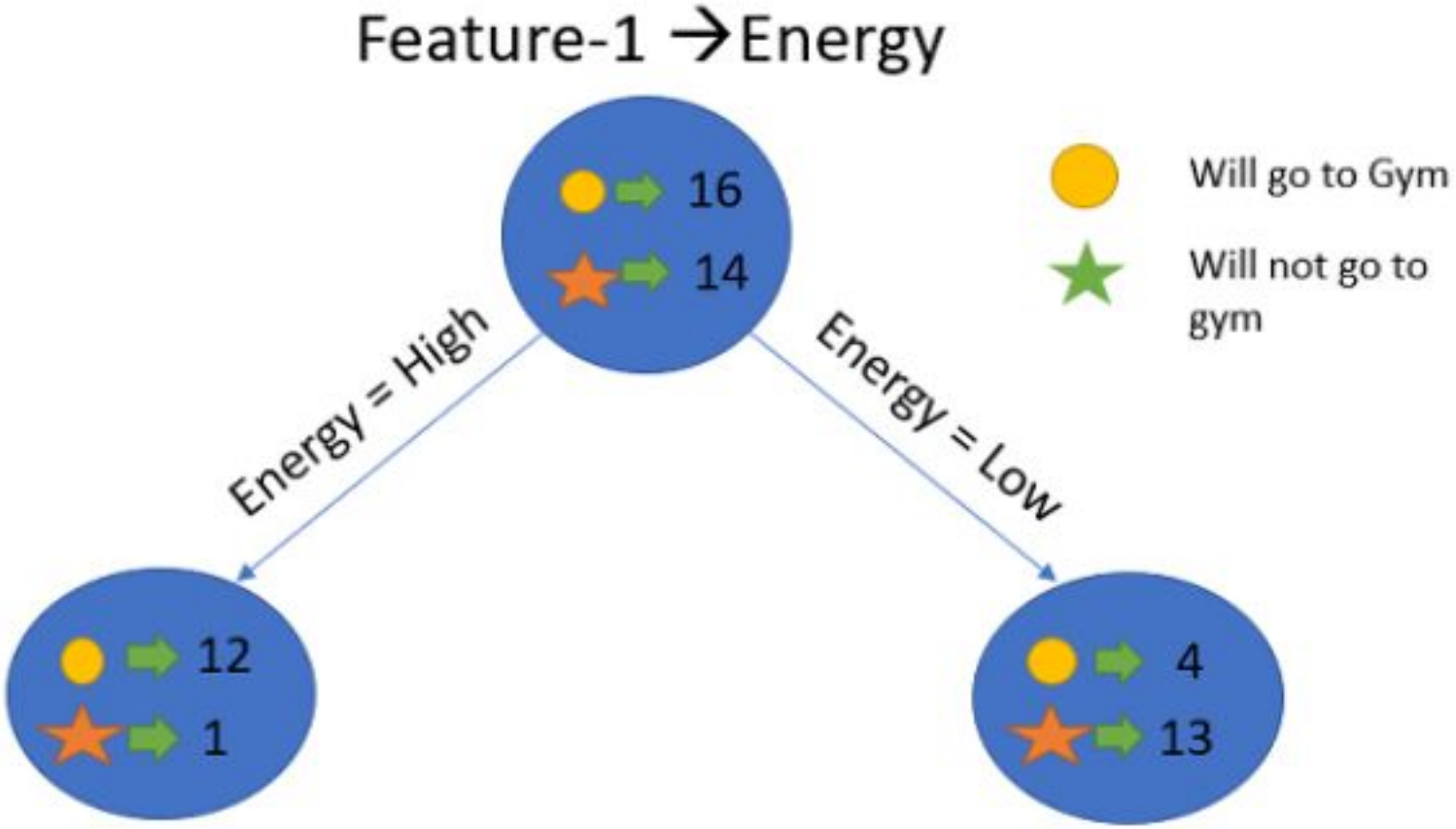
- The left node has low entropy or more purity than right node since left node has a greater number of “yes” and it is easy to decide here.
- Always remember that the higher the Entropy, the lower will be the purity and the higher will be the impurity or uncertainty.
- Here by using the entropy we are getting the impurity of a particular node, we don’t know if the parent entropy or the entropy of a particular node has decreased or not.
- For this, we bring a new metric called “Information gain” which tells us how much the parent entropy has decreased after splitting it with some feature.
- Information Gain measures the reduction of uncertainty given some feature and it is also a deciding factor for which attribute should be selected as a decision node or root node.

$$Information\ Gain = E(Y) - E(Y|X)$$

Example:

- Suppose our entire population has a total of 30 instances. The dataset is to predict whether the person will go to the gym or not. Let's say 16 people go to the gym and 14 people don't
- Now we have two features to predict whether he/she will go to the gym or not.
  i. Feature 1 is **"Energy"** which takes two values *"high" and "low"*
  ii. Feature 2 is **"Motivation"** which takes 3 values *"No motivation", "Neutral" and "Highly motivated".*
- Let's see how our decision tree will be made using these 2 features. We'll use information gain to decide which feature should be the root node and which feature should be placed after the split.

Let’s try with energy feature first as a root:



$$Information\ Gain\ =\ E(Y)\ -\ E(Y|X)$$

$$E(Parent) = -\left(\frac{16}{30}\right)\log_2\left(\frac{16}{30}\right) - \left(\frac{14}{30}\right)\log_2\left(\frac{14}{30}\right) \approx 0.99$$

$$E(Parent|Energy = "high") = -\left(\frac{12}{13}\right)\log_2\left(\frac{12}{13}\right) - \left(\frac{1}{13}\right)\log_2\left(\frac{1}{13}\right) \approx 0.39$$

$$E(Parent|Energy = "low") = -\left(\frac{4}{17}\right)\log_2\left(\frac{4}{17}\right) - \left(\frac{13}{17}\right)\log_2\left(\frac{13}{17}\right) \approx 0.79$$

To see the weighted average of entropy of each node we will do as follows:

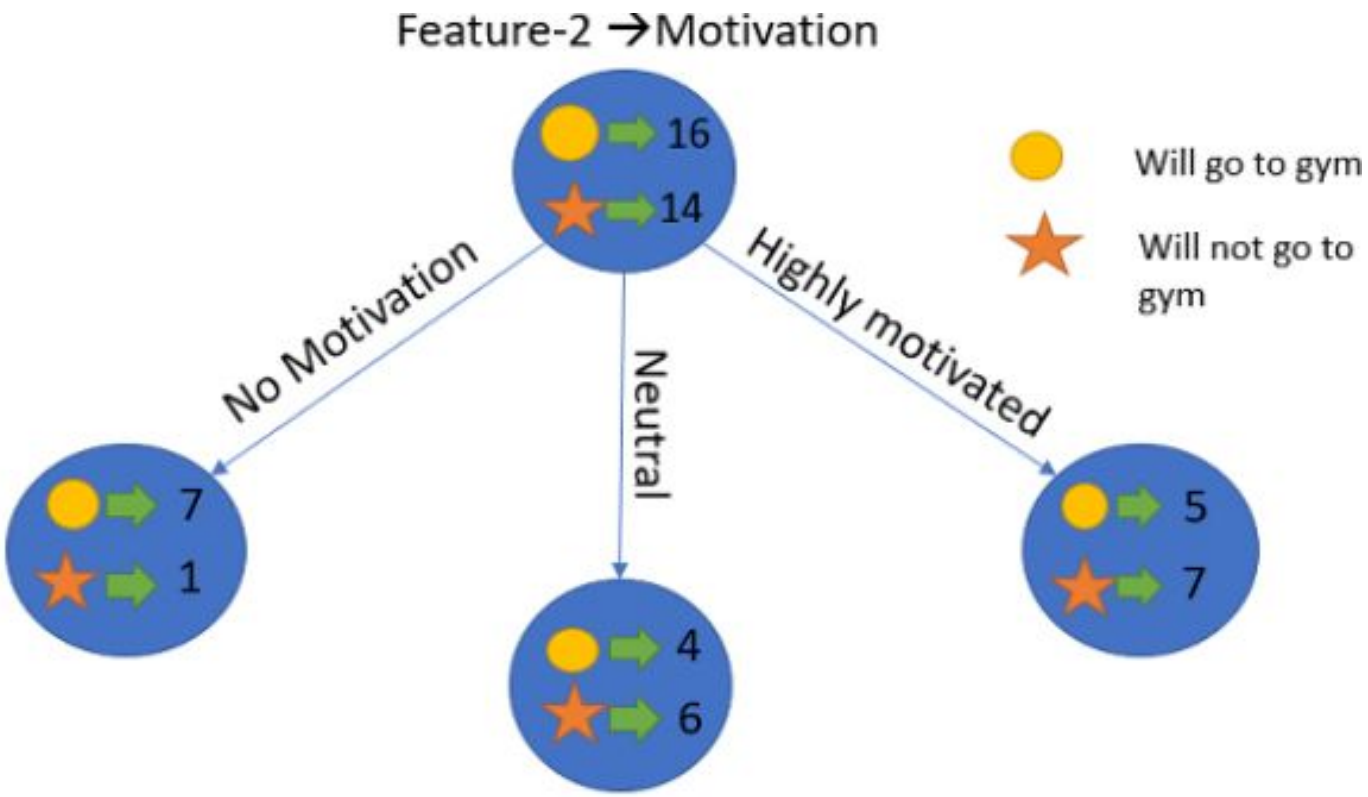
$$E(Parent|Energy) = \frac{13}{30} * 0.39 + \frac{17}{30} * 0.79 = 0.62$$

Now we have the value of E(Parent) and E(Parent|Energy), information gain will be:

$$\begin{aligned} Information\ Gain &= E(parent) - E(parent|energy) \\ &= 0.99 - 0.62 \\ &= 0.37 \end{aligned}$$

Our parent entropy was near 0.99 and after looking at this value of information gain, we can say that the entropy of the dataset will decrease by 0.37 if we make “Energy” as our root node.

- Similarly, we will do this with the other feature “Motivation” as a root and calculate its information gain.



$$E(Parent) = 0.99$$

$$E\Big(Parent|Motivation = "No\ motivation"\Big) = -\left(\frac{7}{8}\right)log_2\left(\frac{7}{8}\right) - \frac{1}{8}\log_2\left(\frac{1}{8}\right) = 0.54$$

$$E(Parent|Motivation = "Neutral") = -\left(\frac{4}{10}\right)log_2\left(\frac{4}{10}\right) - (\frac{6}{10})\log_2\left(\frac{6}{10}\right) = 0.97$$

$$E(Parent|Motivation = "Highly\ motivated") = -\left(\frac{5}{12}\right)log_2\left(\frac{5}{12}\right) - (\frac{7}{12})\log_2\left(\frac{7}{12}\right) = 0.98$$

To see the weighted average of entropy of each node we will do as follows:

$$E(Parent|Motivation) = \frac{8}{30} * 0.54 + \frac{10}{30} * 0.97 + \frac{12}{30} * 0.98 = 0.86$$

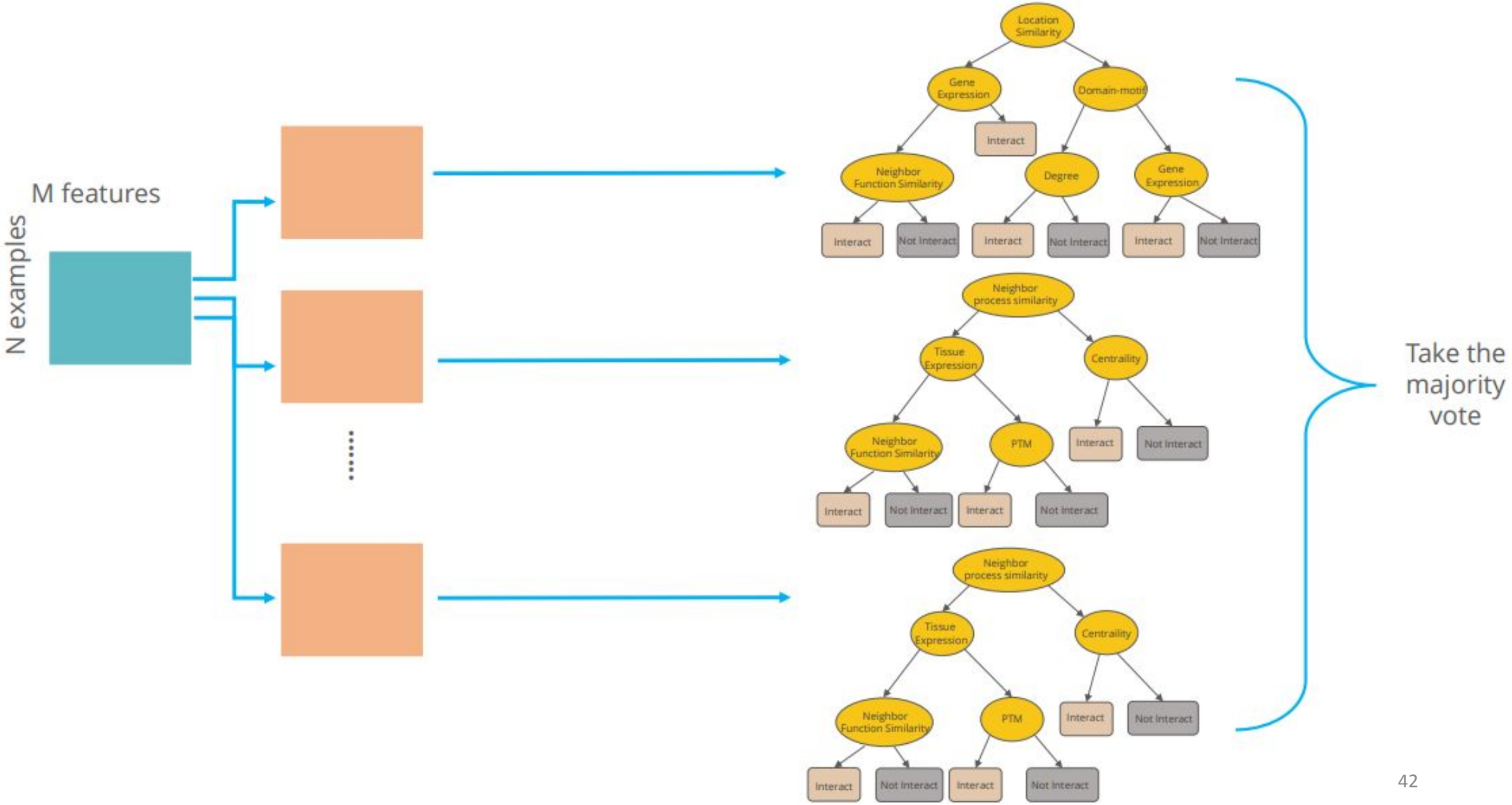
$$E(Parent|Motivation) = \frac{8}{30} * 0.54 + \frac{10}{30} * 0.97 + \frac{12}{30} * 0.98 = 0.86$$

Now we have the value of E(Parent) and E(Parent|Motivation), information gain will be:

$$\begin{aligned} Information\ Gain &= E(Parent) - E(Parent|Motivation) \\ &= 0.99 - 0.86 \\ &= 0.13 \end{aligned}$$

- We now see that the “Energy” feature gives more reduction which is 0.37 than the “Motivation” feature.
- Hence, we will select the feature which has the highest information gain and then split the node based on that feature.
- In this example “Energy” will be our root node and we’ll do the same for sub-nodes.
- Here we can see that when the energy is “high” the entropy is low and hence we can say a person will definitely go to the gym if he has high energy, but what if the energy is low? We will again split the node based on the new feature which is “Motivation”.

Random Forest Classifier



Random Forest: Combination of multiple models

e.g.- Decision Trees for Tennis Playing Prediction

Decision Tree 1:
Outlook: Sunny,
Temperature: 28°C,
Humidity: 65%

Prediction: Yes

Decision Tree 2:
Outlook: Sunny

Prediction: No

Decision Tree 3:
Temperature: 28°C,
Humidity: 65%

Prediction: Yes

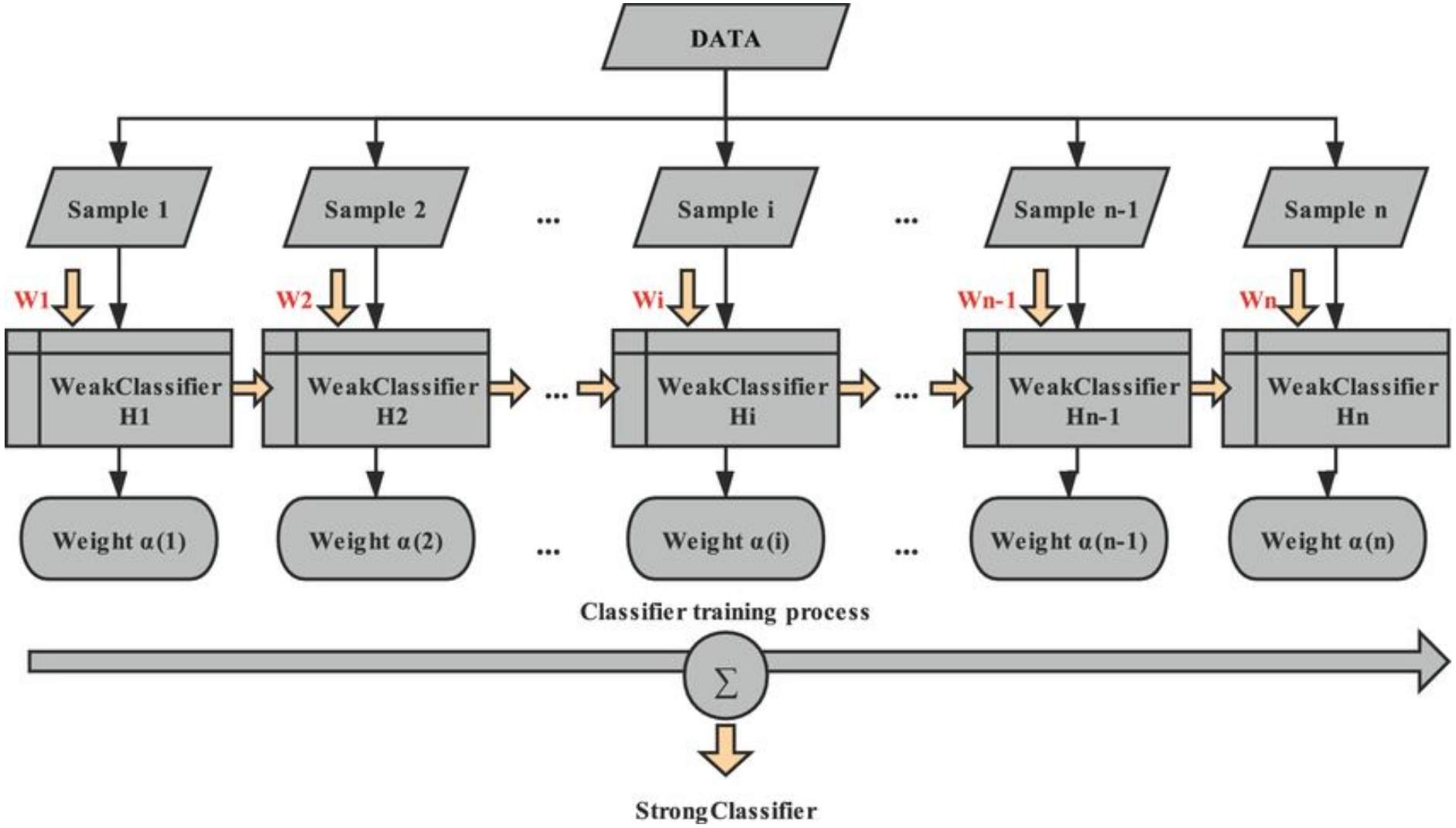
'Yes': 2 votes
'No': 1 vote

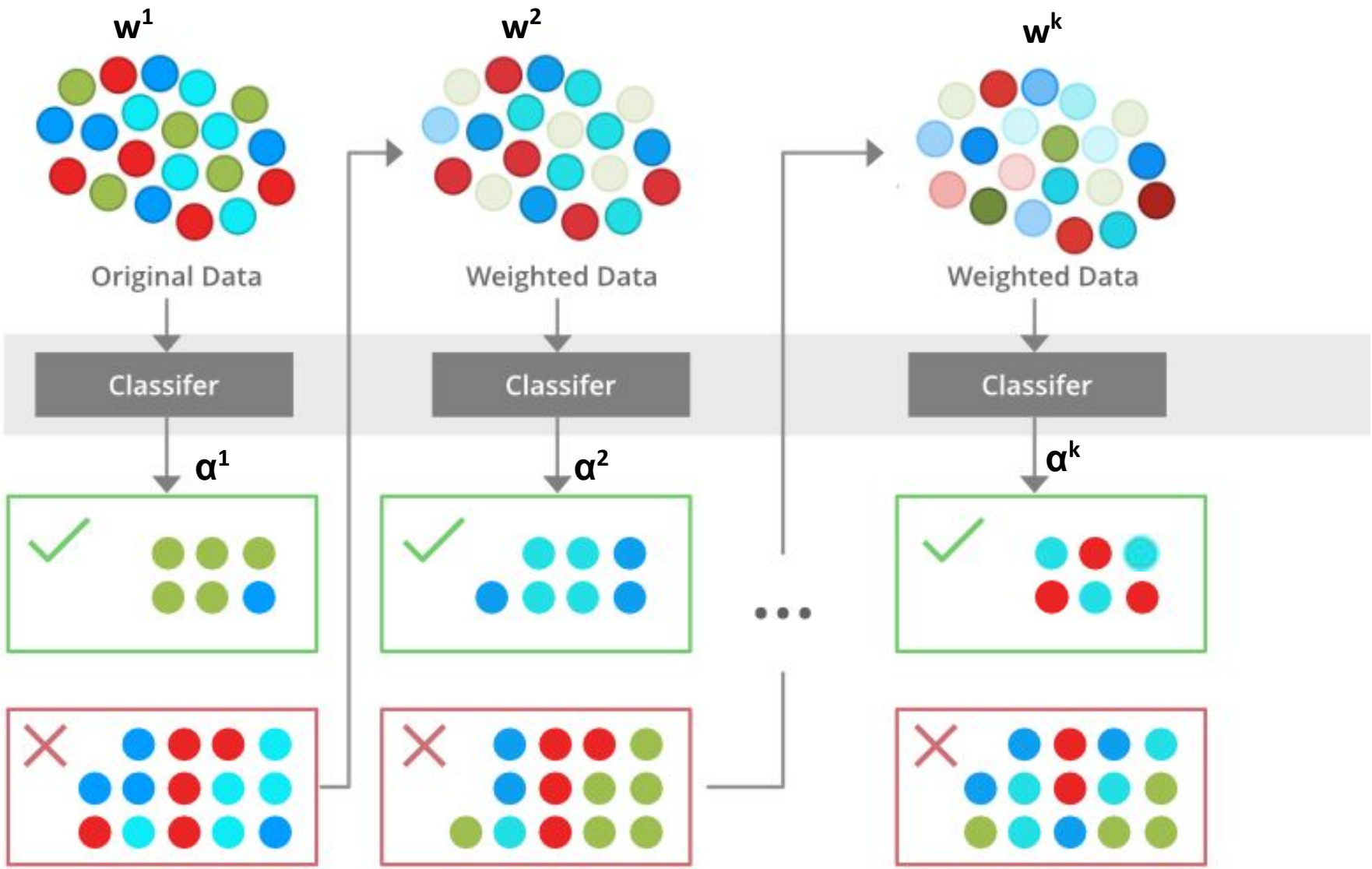
predict that the person will play tennis.

AdaBoost Algorithm: Ensemble Learning

Introduction and Explanation

AdaBoost Algorithm Flowchart



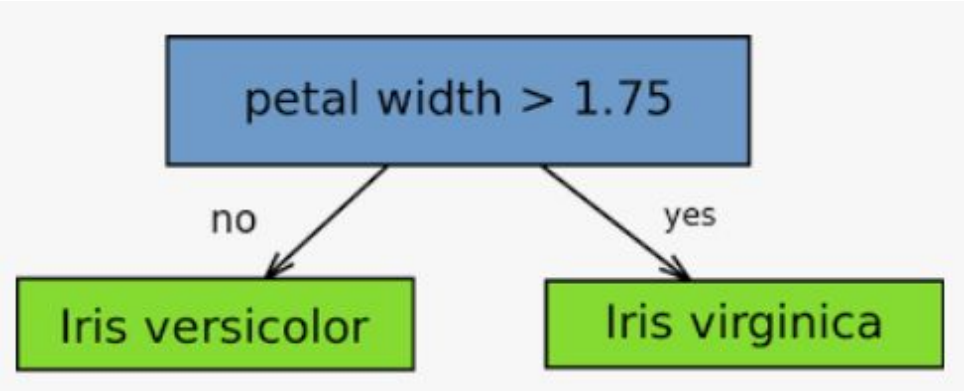

w^1
w^2
w^k
Original Data
Weighted Data
Weighted Data
Classifer
Classifer
Classifer
α^1
α^2
α^k
...

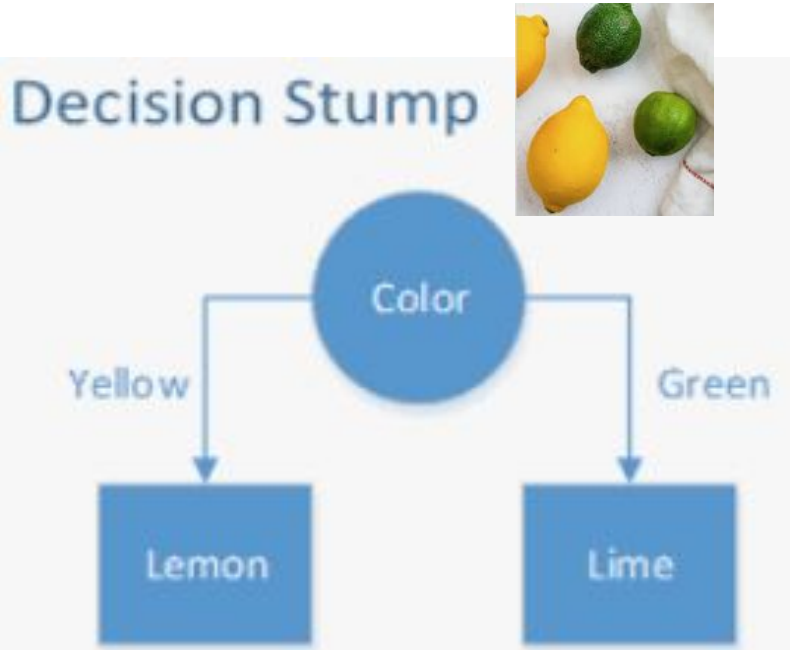
What is AdaBoost Algorithm?

- AdaBoost, short for Adaptive Boosting, is a powerful ensemble learning algorithm used mainly for classification problems.
- It can also be extended to **multi-class classification** using approaches like One-Vs-All.
- It works by combining multiple weak classifiers to create a strong classifier.
- A weak classifier is one that performs slightly better than random guessing, such as a decision stump (a one-level decision tree), which is often used as the base classifier in AdaBoost.

Decision stump

- A decision stump is a simple decision tree model that consists of a single decision node and two leaf nodes. It is the simplest form of decision tree, typically used as a weak classifier in ensemble methods like AdaBoost.





Decision stump continued…

1. **Decision Node**: A decision stump starts with a single decision node, which examines the value of a single feature (attribute) in the dataset. It applies a threshold to this feature and makes a binary decision based on whether the feature value is above or below the threshold.
2. **Leaf Nodes**: The decision node leads to two leaf nodes, each representing one of the binary outcomes (e.g., class labels in a classification problem).

Decision stump continued…

2. **Splitting Criterion**: The decision stump determines the best feature and threshold to split the data based on some criterion, typically aiming to minimize impurity or maximize information gain. However, this choice is limited to a single feature and threshold combination here.
3. **Prediction**: To make predictions, the decision stump applies the decision rule at the decision node to new instances. It classifies instances based on whether their feature values satisfy the condition defined by the decision node, leading them to one of the leaf nodes where the class label is assigned.

Working of AdaBoost

1. Initialize weights: Initially, each data point is given an equal weight. For a dataset with N data points, each weight $\mathbf{w_i}$ is initialized as 1/N.

2. Build a weak classifier: A weak classifier is trained on the dataset. It aims to classify the data better than random chance.

3. Evaluate the weak classifier: The weak classifier's performance is evaluated, so that correctly classified and misclassified data points are also assigned with some weights ($\boldsymbol{\alpha_i}$).

4. Update weights: Misclassified points receive higher weights, while correctly classified points receive lower weights, making misclassified points more influential in the next iteration. This makes the algorithm focus more on the data points that are difficult to classify

Weight updation:

https://blog.paperspace.com/adaboost-optimizer/

The update formula for the weight of data point i in iteration t is:

$$w_i^{(t+1)} = w_i^{(t)} \times \exp(-\alpha^{(t)} y_i h^{(t)}(x_i))$$

where:

- $\alpha^{(t)}$ is the weight assigned to the weak classifier in iteration t.
- y_i is the true label of data point i (1 for positive class, -1 for negative class).
- $h^{(t)}(x_i)$ is the prediction of the weak classifier for data point i in iteration t.
- $\exp(-\alpha^{(t)} y_i h^{(t)}(x_i))$ is a factor that increases the weight of misclassified points and decreases the weight of correctly classified points.

- When $y_i = h^t(x_i)$, $W_i^{(t+1)}$ will decrease otherwise increase.

- After updating the weights, AdaBoost normalizes them so that they sum up to 1, ensuring that they represent a valid probability distribution.
- The weights of correctly classified points may be decreased, although this isn't always necessary (AdaBoost doesn’t explicitly reduce these weights), but the normalization step ensures that the weights sum up to 1 (influence **indirectly** reduce weights for correctly classified samples).
- After each iteration, the weights of **misclassified samples increase**, making them more likely to be selected in the next iteration.

$w_i^{(t)}$:

- $w_i^{(t)}$ is the weight assigned to each individual data point i at iteration t.
- Initially, all data points have equal weights, typically set to $\frac{1}{N}$, where N is the total number of data points.
- In subsequent iterations, these weights are updated based on the performance of the weak classifiers. Misclassified points are assigned higher weights to make them more influential in the next iteration, while correctly classified points may have their weights decreased (although this isn't always necessary due to the normalization step).
- $w_i^{(t)}$ is used to adjust the importance of individual data points in the training process.

$\alpha^{(t)}$:

- $\alpha^{(t)}$ is the weight assigned to the weak classifier in iteration t.
- It represents the contribution or importance of the weak classifier to the final combined classifier.
- $\alpha^{(t)}$ is calculated based on the performance of the weak classifier in minimizing the weighted error.
- A higher $\alpha^{(t)}$ indicates that the weak classifier performs well on the training data and will have more influence in the final combined classifier.
- $\alpha^{(t)}$ is used to determine the overall weight of each weak classifier in the ensemble.

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right)$$

ϵ is weighted error to be minimized

Misclassification Indicator:

For each data point i, a misclassification indicator $e_i^{(t)}$ is defined as follows:

- If the weak classifier misclassifies data point i in iteration t, then $e_i^{(t)} = 1$.
- If the weak classifier correctly classifies data point i in iteration t, then $e_i^{(t)} = 0$.

Weighted error calculation

The weighted error $\epsilon^{(t)}$ of the weak classifier in iteration t is calculated as the sum of the weights of the misclassified data points, normalized by the sum of all weights:

$$\epsilon^{(t)} = \frac{\sum_{i=1}^{N} w_i^{(t)} \cdot e_i^{(t)}}{\sum_{i=1}^{N} w_i^{(t)}}$$

where:

- N is the total number of data points.
- $w_i^{(t)}$ is the weight assigned to data point i in iteration t.
- $e_i^{(t)}$ is the misclassification indicator for data point i in iteration t.

Weighted error calculation continued…

- This calculation essentially averages the misclassification indicators over all data points, weighted by their respective weights.
- **Interpretation**: A lower weighted error indicates that the weak classifier performs better on the training data, as it misclassifies fewer heavily weighted data points.
- Conversely, a higher weighted error suggests that the weak classifier's performance is poor, and it struggles to correctly classify the more influential data points.
- The goal of AdaBoost is to minimize the weighted error of the weak classifiers by iteratively adjusting their parameters and weights.

- In AdaBoost, the classifier weight α is determined based on the weighted error ϵ of the weak classifier.

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right)$$

- If ϵ = 0.5, α = 0, meaning the weak classifier is useless.
- If ϵ > 0.5, the weak classifier is worse than random.
- If the classifier is perfect (i.e. ϵ=0), then α = ∞. Which means the weak classifier is extremely strong, and AdaBoost assigns it a very high weight.
- ϵ should be between 0 to 0.5 (lesser is better).

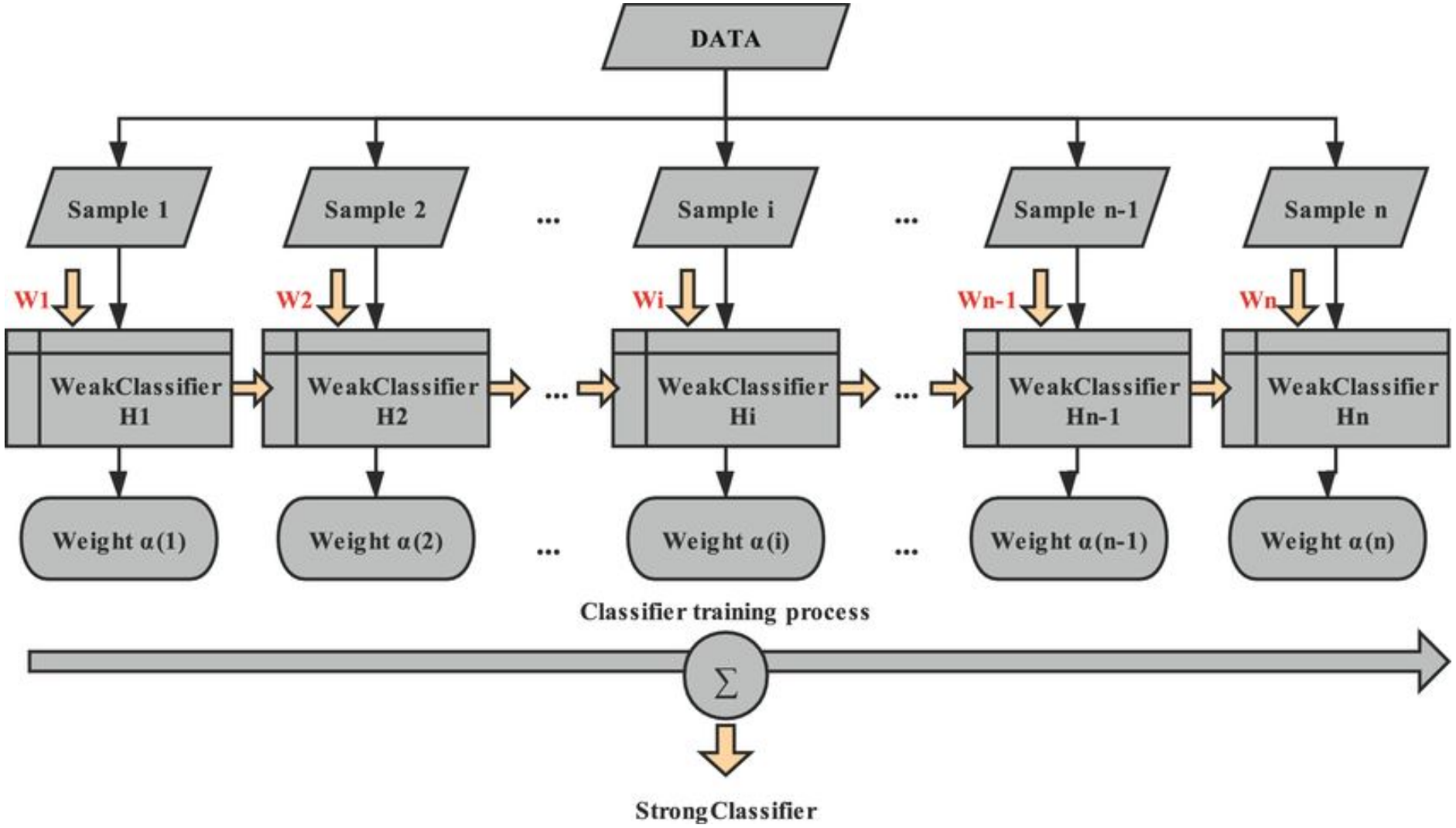
5. Build another weak classifier: Another weak classifier is trained on the updated dataset, giving more weight to the previously misclassified points.

6. Repeat: Steps 3-5 are repeated for a predetermined number of iterations or until the model performs well on the training data.

7. Combine weak classifiers: Each weak classifier is assigned a weight based on its accuracy. The final classification is determined by a weighted sum of the weak classifiers.

8. Final model: The combined weak classifiers form the final strong classifier, which can then be used to make predictions on new data.

AdaBoost Algorithm Flowchart



- AdaBoost helps in ensemble learning by combining the outputs of multiple weak classifiers to create a single strong classifier.

- Benefits:
    1. Improved accuracy
    2. Reduced overfitting
    3. Versatility

Example: AdaBoost in Binary Classification

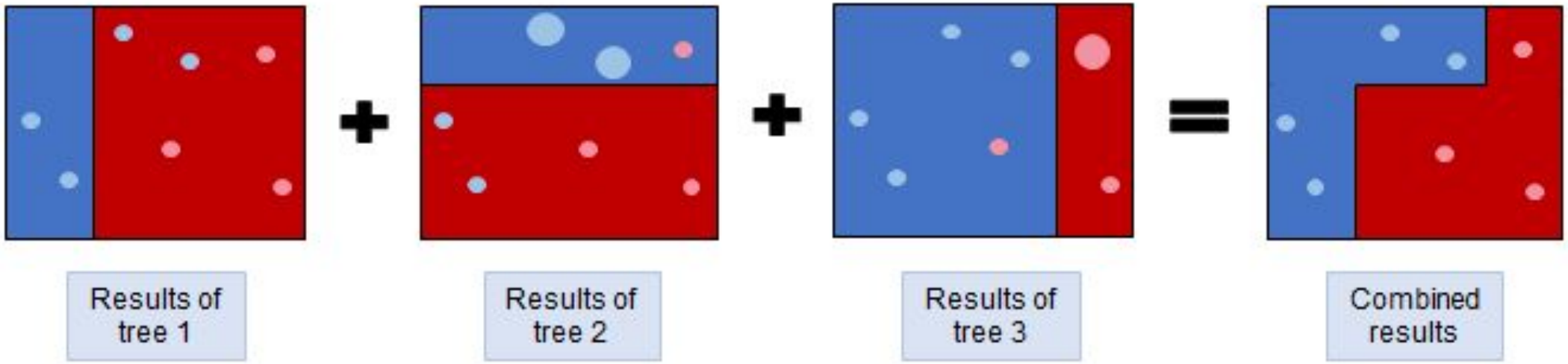
- Let's consider a binary classification problem where we want to classify whether a patient has a particular disease based on some medical test results. We have a dataset with features such as age, blood pressure, and cholesterol levels.

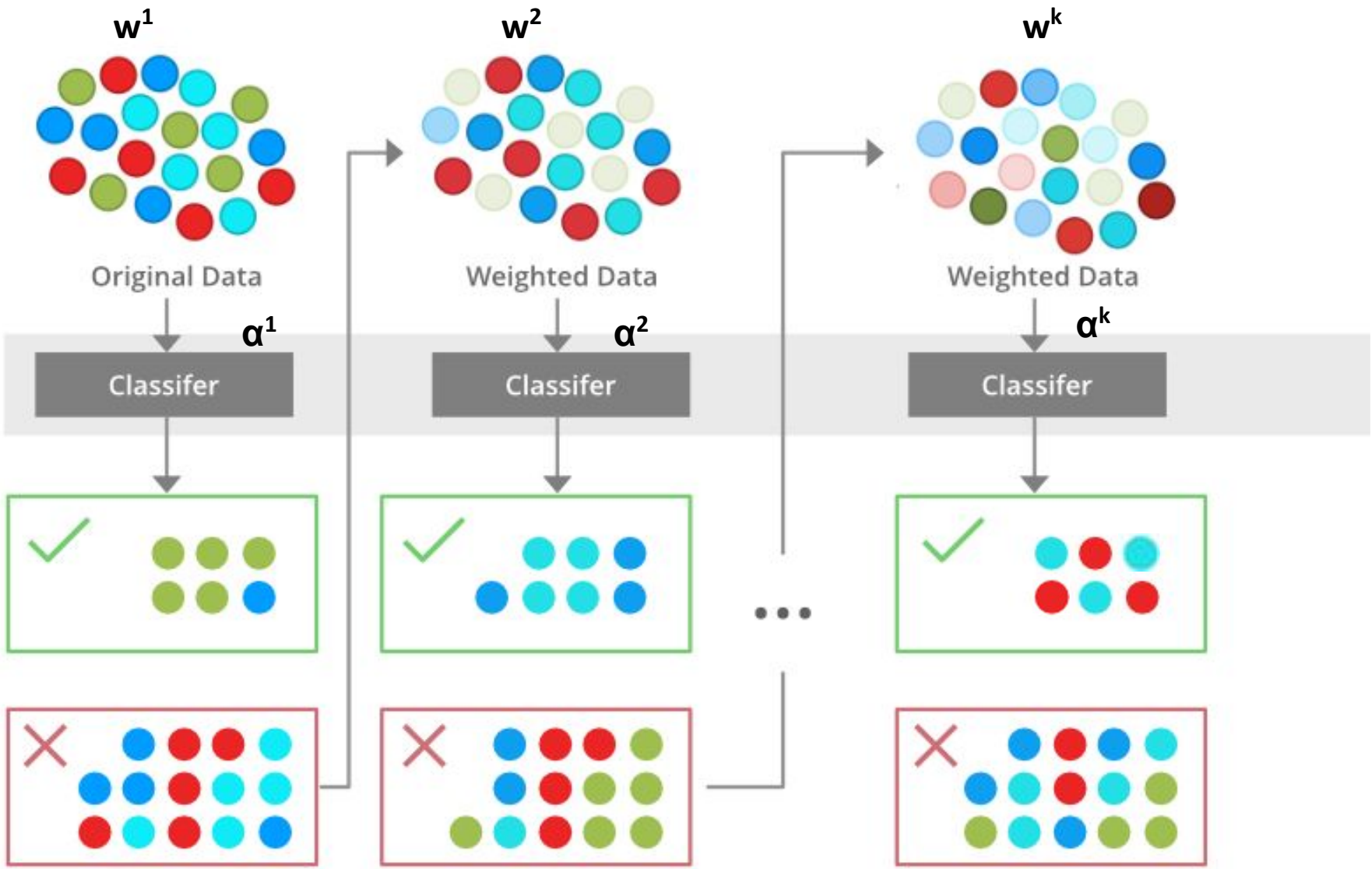
1. Initially, all data points are given equal weight.
2. We train a decision stump (weak classifier) on the data.
3. Evaluate the weak classifier. Suppose it misclassifies some patients.
4. Update the weights of the misclassified patients, giving them higher weights.
5. Train another weak classifier, giving more weight to the misclassified patients.
6. Repeat steps 3-5 for a few iterations.
7. Combine the weak classifiers, assigning higher weights to more accurate classifiers.
8. The final model is a strong classifier that can predict whether a patient has the disease based on their medical test results.

Example: AdaBoost in Binary Classification

AdaBoost:

- Combining **weak learners** (decision trees)
- Assigning **weights to incorrect values**
- **Sequential tree growing** considering past mistakes



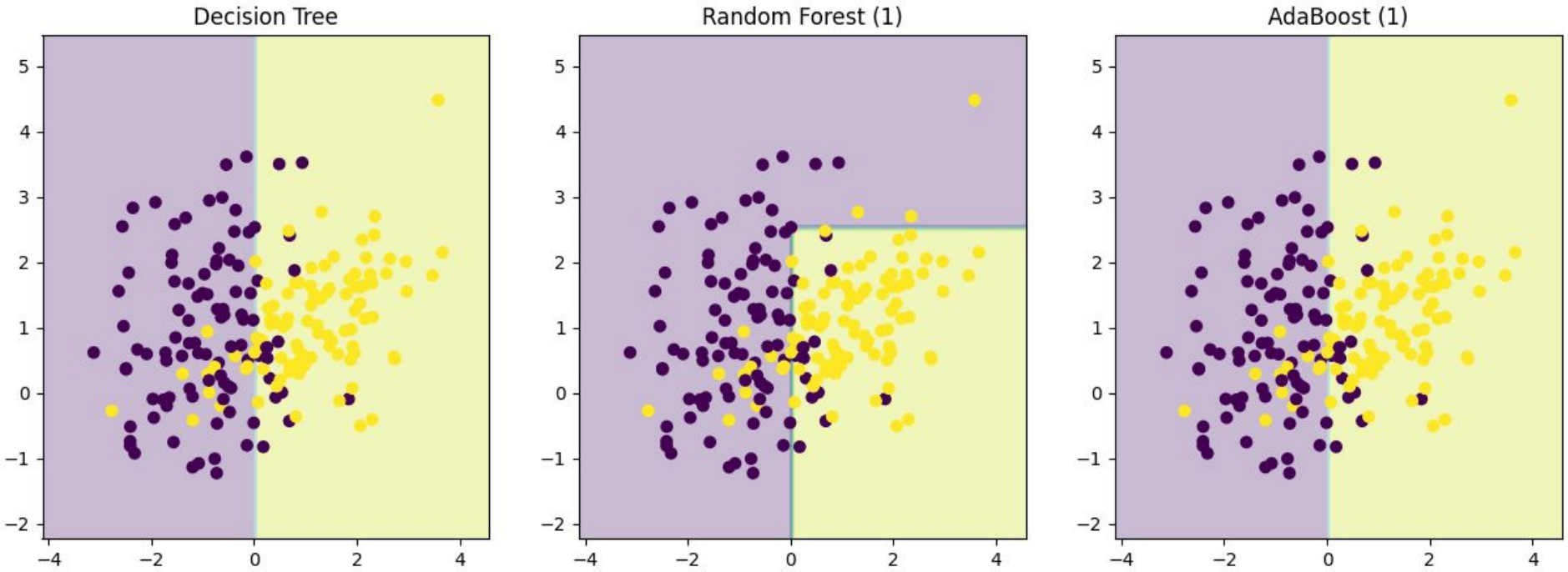

w^1
w^2
w^k
Original Data
Weighted Data
Weighted Data
α^1
α^2
α^k
Classifer
Classifer
Classifer
...

Comparison

Decision Tree: Single model

Random Forest: Parallel ensemble

Adaboost



Stacking models

- Also known as stacked generalization
- Combines multiple classification or regression models via a meta classifier or meta regressor.
- Base level model predictions are used as input features for a final meta-model.
- Base level (level-0 models) often consist of different learning algorithms and therefore stacking ensembles are often heterogeneous.
- A single Meta model (level-1 model) is used to learn how to best combine predictions from the contributing models.

Components of Stacking

- BASE MODELS:
  - ✔ Individual ML models
  - ✔ Can be of different type
  - ✔ Trained on the same dataset

- META MODEL (or STACKER):

Higher level single model

  - ✔ Combines predictions of the base models
  - ✔ Inputs can be real values (in case of regression) or Probability like values or class labels in case of classification

- The stacking process may also involve
  - ✔ splitting the training data into multiple folds,
  - ✔ training base models on **different folds**,
  - ✔ generating predictions from these models,
  - ✔ and then training a meta-model on these predictions.

Stacking Process

1. Split the data into two parts viz, a training set and test set. The training data is further split into K-folds just like K-fold cross-validation.

2. A base model (e.g k-NN) is fitted on the K-1 parts and predictions are made for the Kth part.

3. This process is iterated until every fold has been predicted. The predictions from each fold are collected to form an **out-of-fold (OOF) predictions dataset** (for the training set).

4. After cross validation the base model is then fitted on the whole train data set to calculate its performance on the test set.
5. We repeat the last 3 steps for other base models.(e.g SVM, decision tree, neural network etc)

   Each base model generates:

   i. **Out-of-fold predictions** for training.

   ii. **Predictions on the test set**.
6. Predictions from the train set are used as features for the second level model.
7. Second level model is used to make a prediction on the test set.

Benefits of stacking

- Improved performance
- Robustness
- Flexibility

Feature	Stacking	Meta-Learning
Definition	Combines predictions from multiple base models	Learns how to optimize model training itself
Base Models	Traditional models like k-NN, SVM, Decision Tree, etc.	Algorithms that modify their own learning process (e.g., MAML in deep learning)
Training Strategy	Uses out-of-fold predictions as new features for the meta-model	Optimizes the learning algorithm to generalize across tasks
Goal	Improve accuracy by combining weak learners	Improve adaptability of models for unseen tasks
Example	Stacking SVM, Decision Tree, and k-NN with Logistic Regression as the final model	A model that learns to adjust hyperparameters automatically

- **Stacking** is an **ensemble learning** technique that improves accuracy by combining different models.
- **Meta-learning** is a higher-level learning process that helps models learn how to **learn better** across multiple tasks.

AdaBoost Stacking

- AdaBoost can be used as a base model in a stacking ensemble.
- In this scenario, AdaBoost would be trained along with other base models, and their predictions would be combined using a meta-model (e.g., a neural network, random forest, etc.).
- This allows AdaBoost to contribute its predictions along with other models, potentially improving overall performance.

Blending models

- Blending is similar to stacking but simpler. It uses only a holdout (validation) set from the training set to make predictions.
- Instead of using a separate meta-model, blending directly averages the predictions of base models to generate the final prediction.
- The **training data** is usually split into two parts: one part is used to train the base models, and the other part is used to generate predictions from these models. Then, the predictions from the base models are averaged to produce the final prediction.
- Major difference between stacking and blending:
  - ✔ In stacking, the training dataset is split into multiple folds for training base models, and predictions from these models are used to train a meta-model on separate folds.
  - ✔ In blending, the training dataset is typically split into two parts: one for training base models and the other for generating predictions, which are then combined without a meta-model or used to train a meta model.

- **Stacking**: All training samples get predicted (OOF predictions). Train the **meta-model** using these OOF predictions as features.

- **Blending**: Use the **base models' predictions on validation set** as inputs to the meta-model.

Blending with AdaBoost

- AdaBoost predictions can be blended with predictions from other models.
- After training AdaBoost and other models on the training data, their predictions can be averaged or weighted to produce the final prediction.
- Blending is straightforward and can be effective in improving predictive accuracy.

In both stacking and blending, the choice of base models, the method of combining predictions, and the selection of hyperparameters are crucial for achieving optimal performance.

Extreme Gradient Boosting (XGBoost)

- XGBoost, or eXtreme Gradient Boosting, is a machine learning algorithm under ensemble learning.
- It is trendy for supervised learning tasks, such as regression and classification.
- XGBoost builds a predictive model by combining the predictions of multiple individual models, often decision trees, in an iterative manner.
- The algorithm works by sequentially adding weak learners to the ensemble, with each new learner focusing on correcting the errors made by the existing ones.

- It uses a gradient descent optimization technique to minimize a predefined loss function during training.
- It is the execution of gradient boosted decision trees that is designed for high speed and performance.
- Even when it comes to machine learning competitions and hackathon, XGBoost is one of the excellent algorithms that is picked initially for structured data.
- It has proved its determination in terms of speed and performance.
- XGBoost is widely used in various domains due to its high predictive performance and versatility across different datasets.

Features

- **Regularization:** through both L1 and L2 regularization.
- **Handling sparse data:** Missing values or data processing steps like one-hot encoding make data sparse. XGBoost Classifier incorporates a sparsity-aware split finding algorithm to handle different types of sparsity patterns in the data.
- **Weighted quantile sketch:**
  - The standard quantile sketch algorithm assumes that each data point contributes equally, so it does not work well when some points are more important than others (not equipped to handle weighted data).
  - The **split point** is the **threshold at a node.**
  - XGBoost has a distributed weighted quantile sketch algorithm to effectively handle weighted data.

Features

- **Block structure for parallel learning:** For faster computing, XGBoost Classifier can make use of multiple cores on the CPU. This is possible because of a block structure in its system design. Data is sorted and stored in in-memory units called blocks. Unlike other algorithms, this enables the data layout to be reused by subsequent iterations, instead of computing it again. This feature also serves useful for steps like split finding and column sub-sampling.
- **Cache awareness:** In XGBoost, non-continuous memory access is required to get the gradient statistics by row index. Hence, XGBoost has been designed to make optimal use of hardware. This is done by allocating internal buffers in each thread, where the gradient statistics can be stored
- **Out-of-core computing:** This feature optimizes the available disk space and maximizes its usage when handling huge datasets that do not fit into memory

XGBoost Model Benefits and Attributes

1. **High accuracy:** Xgboost Classifier is known for its accuracy and has been shown to outperform other machine learning algorithms in many predictive modeling tasks.
2. **Scalability:** Handle large datasets with millions of rows and columns.
3. **Efficiency:** Can quickly train models on large datasets.
4. **Flexibility:** It supports a variety of data types and objectives, including regression, classification, and ranking problems.
5. **Regularization:** It incorporates regularization techniques to avoid overfitting and improve generalization performance.
6. **Interpretability:** It provides feature importance scores that can help users understand which features are most important for making predictions.
7. **Open-source:** XGBoost Model is an open-source library that is widely used and supported by the data science community.

Difference between XGBoost and Random Forest:

- **Algorithm Type:** XGBoost Classifier works by improving mistakes from previous trees, while Random Forest builds trees independently.
- **Handling of Weak Learners:** XGBoost corrects errors sequentially, while Random Forest combines predictions of independently built trees.
- **Regularization:** XGBoost prevents overfitting using techniques like L1 and L2 regularization, whereas Random Forest usually doesn't.
- **Performance:** XGBoost often performs better on structured data but needs more tuning. Random Forest is simpler and less prone to overfitting.

Gradient Boosting vs. Adaptive Boosting (AdaBoost)

Aspect	AdaBoost	XGBoost
Boosting Approach	Adjusts weights of misclassified samples in each iteration	Fits new models to the residuals (errors) of the previous models
Weak Learners	Uses shallow decision trees with limited splits	Uses decision trees, often deeper and more complex
Handling of Difficult Samples	Increases weights of misclassified instances	Minimizes loss by optimizing residuals
Regularization	No inherent regularization	Includes L1 and L2 regularization to prevent overfitting
Optimization Method	Focuses on weighted classification errors	Uses gradient boosting to minimize a loss function
Performance on Complex Data	May struggle with highly complex datasets	Performs well on large datasets with complex patterns
Implementation Complexity	Simpler and easier to implement	More complex but highly optimized
Scalability	Works well for smaller datasets	Highly scalable, optimized for large datasets
Computational Efficiency	Generally faster but less optimized	Uses parallelization and efficient memory usage for better performance

- It uses a gradient descent algorithm that is the reason it is called a “**Gradient Boosting Algorithm**”. Weather classification or regression methods are supported for both types of predictive modelling problems.

State-of-the-art prediction performance
  - Won Netflix Challenge
  - Won numerous KDD Cups
  - Industry standard

Method	Minimize Bias?	Minimize Variance?	Other Comments
Bagging	Complex model class. (Deep DTs)	Bootstrap aggregation (resampling training data)	Does not work for simple models.
Random Forests	Complex model class. (Deep DTs)	Bootstrap aggregation + bootstrapping features	Only for decision trees.
Gradient Boosting (AdaBoost)	Optimize training performance.	Simple model class. (Shallow DTs)	Determines which model to add at run-time.
Ensemble Selection	Optimize validation performance	Optimize validation performance	Pre-specified dictionary of models learned on training set.
XGBoost	Optimizes training performance with gradient boosting. Uses second-order Taylor expansion for better updates.	Includes L1 & L2 regularization to reduce overfitting (better variance control than AdaBoost)	More efficient than AdaBoost. Supports parallelization and handles missing values.

…and many other ensemble methods as well.